Simulacija Okeana Pomoću Brze Fourierove Transformacije (FFT)

Poboljšanje izgleda i početak paralelizacije na CPU

Drugi zadatak

Zadatak za ovu sedmicu je bio usavršavanje izgleda okeanskih valova, dodavanje detalja i efekta pjene. Nakon toga započet je rad na paraleliraciji na procesoru.

1 Osvrt na prijašnju verziju

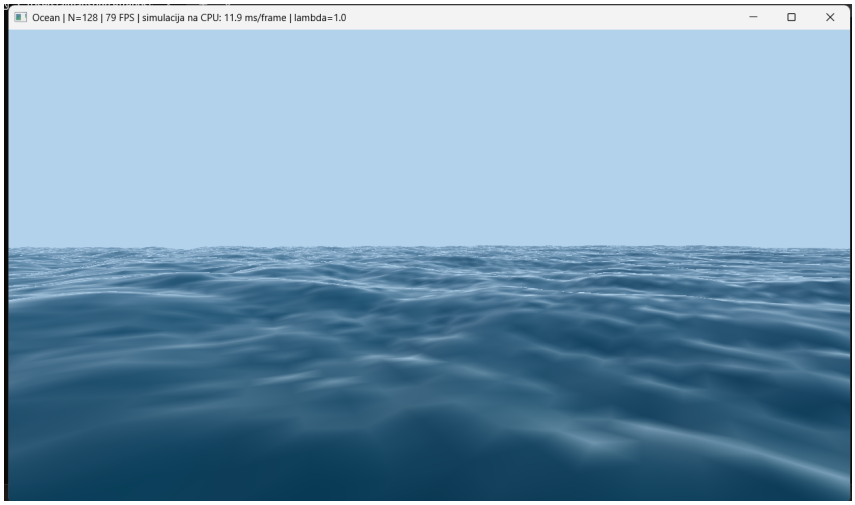
Prva verzija implementacije (jedna kaskada, $N = 256$, $L = 250\,\text{m}$) imala je sljedeće nedostatke:

1. **Nerealne visine talasa zbog neodgovarajuće amplitude.** Pri prvim pokretanjima konstanta amplitude Phillipsovog spektra bila je $A = 1$, pa su visine bile u rasponu od oko $-6000\,\text{m}$ do $+8000\,\text{m}$ umjesto nekoliko metara. Amplituda je prvo ručno smanjena ($A = 5 \cdot 10^{-7}$), a zatim je uvedena kalibracija iz brzine vjetra (odjeljak 3.1.1).

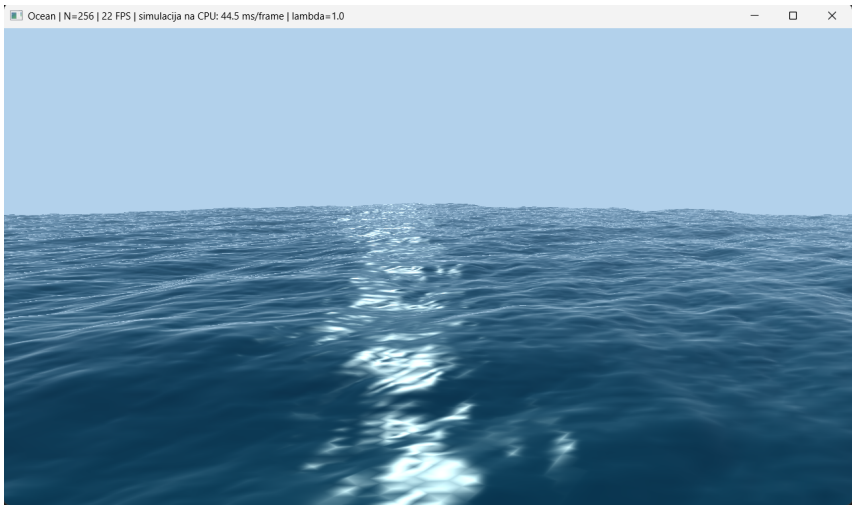
2. **Pet zasebnih 2D IFFT-ova po frejmu.** Visina h, nagibi $\partial h/\partial x$ i $\partial h/\partial z$ te choppy pomaci D_x i D_z računali su se kroz pet odvojenih 2D IFFT poziva. Za $N = 256$ IFFT je trajao $45.74\,\text{ms}$, što je oko 93% ukupnog vremena frejma ($49.30\,\text{ms}$), pa je on bio glavni trošak.

3. **Jedna kaskada i bez pjene.** Simulirana je samo jedna kaskada ($N = 256$, $L = 250\,\text{m}$), pa je najfiniji razmak tačaka iznosio $0.98\,\text{m}$. Vrhovi talasa su bili obli, bez vizuelnog prikaza lomljenja (pjene).

Dinamička alokacija pomoćnih nizova u svakom frejmu nije otklonjena ni u novoj verziji (vidi odjeljak 5.6).



Slika 1: Prikaz izgleda stare verzija sa N=128.



Slika 2: Prikaz izgleda stare verzija sa N=256.

2 Unapređenja Implementacije i Optimizacija Performansi

U cilju postizanja većeg vizuelnog realizma i rada u realnom vremenu pri visokim rezolucijama, osnovni algoritam je značajno nadograđen u nekoliko ključnih segmenata.

2.1 Dvostruka Spektralna Kaskada (Multi-Cascade Spectrum)

Umjesto simulacije jedne mreže dimenzija $N \times N$, uvedene su dvije komplementarne spektralne kaskade:

1. **Geometrijska kaskada ($N_0 = 256, L_0 = 400$ m):** Pokriva niske frekvencije ($k < k_{\text{split}}$) i izračunava osnovnu 3D geometriju površine (visine, choppy pomake i glavne nagibe).
2. **Kaskada detalja ($N_1 = 128, L_1 = 40$ m):** Pokriva visoke frekvencije ($k > k_{\text{split}}$) i izračunava samo nagibe sitnih vala. Ovi nagibi se upisuju u `RG32F` GPU teksturu sa automatski generisanim mipmapama.

Za glatki prelaz snage između kaskada bez dupliranja energije primjenjuje se težinska funkcija $w(k)$:

$$w_0(k) = \cos\left(\frac{\pi}{2} t(k)\right), \qquad w_1(k) = \sin\left(\frac{\pi}{2} t(k)\right) \tag{1}$$

gdje je $t(k) = \text{smoothstep}\left(\frac{k - k_{\text{lo}}}{k_{\text{hi}} - k_{\text{lo}}}\right)$. Najfiniji razmak tačaka ovim je smanjen sa 0.98 m na 0.31 m, što odgovara efektivnoj rezoluciji mreže od oko 1280×1280 tačaka.

2.2 Pakovani 2D IFFT (Algoritam sa 3 Transformacije)

Pošto su prostorna polja visina i nagiba realni brojevi, njihovi spektri posjeduju Hermitovsku simetriju $\tilde{h}^*(-\mathbf{k}) = \tilde{h}(\mathbf{k})$. Dva realna polja $A(\mathbf{x})$ i $B(\mathbf{x})$ pakuju se u jedan kompleksni spektar:

$$Z(\mathbf{k}) = \tilde{A}(\mathbf{k}) + i\tilde{B}(\mathbf{k}) \implies \text{IFFT}[Z(\mathbf{k})] = A(\mathbf{x}) + iB(\mathbf{x}) \tag{2}$$

Umjesto 5 inverznih 2D FFT-ova po frejmu, iz izvode se samo 3 transformacije:

1. Visina h: $\text{IFFT}[\tilde{h}(\mathbf{k}, t)]$
2. Nagibi: $\text{IFFT}[(ik_x - k_z)\tilde{h}(\mathbf{k}, t)] \implies \frac{\partial h}{\partial x} + i\frac{\partial h}{\partial z}$
3. Choppy pomaci: $\text{IFFT}\left[\left(\frac{k_z}{k} - i\frac{k_x}{k}\right)\tilde{h}(\mathbf{k}, t)\right] \implies D_x + iD_z$

2.3 Simulacija Pjene pomoću Jacobijana

Lomljenje valova na vrhovima detektuje se preko determinante Jacobijana preslikavanja deformisane mreže $\mathbf{x}' = \mathbf{x} + \lambda \mathbf{D}(\mathbf{x}, t)$:

$$J = \det(\nabla \mathbf{x}') = (1 + \lambda D_{xx})(1 + \lambda D_{zz}) - \lambda^2 D_{xz} D_{zx} \tag{3}$$

gdje se parcijalni izvodi pomaka $D_{xx}, D_{xz}, D_{zx}, D_{zz}$ računaju centralnim konačnim razlikama. Pjena nastaje kada je $J <$ bias i eksponencijalno opada u vremenu sa konstantom τ:

$$\text{foam}(t) = \max\left(\text{foam}_{t-\Delta t} \cdot e^{-\Delta t/\tau},\ \text{clamp}((\text{bias} - J) \cdot \text{gain}, 0, 1)\right) \tag{4}$$

3 Analiza Praktične Implementacije i Ekstraktovanih Rezultata

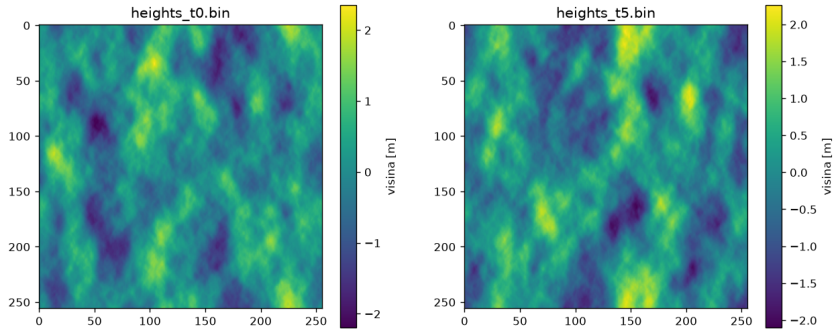
U sklopu praktičnog dijela projekta realizovana je kompletna C++ biblioteka za simulaciju (`ocean.h`, `fft.h`), modul za testiranje i izvoz u binarne datoteke (`test_ocean.cpp`, `test_fields.cpp`), Python skripta za vizuelnu analizu niza polja (`plot_heights.py`), te interaktivni 3D OpenGL vizualizator (`viewer.cpp`).

3.1 Verifikacija Polja u Pythonu (Binarna Analiza)

Za provjeru matematičke korektnosti i vremenske evolucije, C++ modul `test_ocean.cpp` i `test_fields.cpp` izvoze stanja simulirane mreže u binarne datoteke formata `float32`. Učitavanje i prikaz vrši se pomoću Python skripte (`plot_heights.py`).

3.1.1 Vremenska evolucija i provjera visina

Slika 3 prikazuje visinska polja $h(\boldsymbol{x}, t)$ u trenucima $t = 0\,\text{s}$ i $t = 5\,\text{s}$ na mreži $N = 256$ nakon kalibracije amplitude.



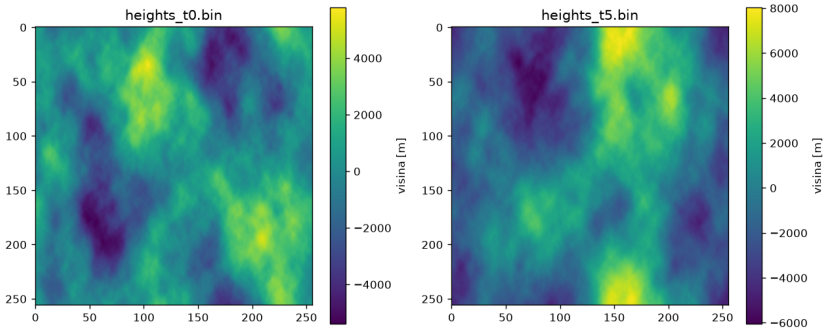
Slika 3: Visinsko polje u $t = 0\,\text{s}$ (lijevo) i $t = 5\,\text{s}$ (desno) nakon kalibracije amplitude (visine u rasponu od oko $-2.1\,\text{m}$ do $+2.3\,\text{m}$).

Detekcija nerealnih visina i kalibracija amplitude: Pri prvim pokretanjima visine su imale nerealne raspone od oko $-6000\,\text{m}$ do $+8000\,\text{m}$ (slika 4). Skaliranje IFFT-a je ispravno: implementirani 2D IFFT dijeli rezultat sa N^2, pa se on množi sa N^2 da bi se dobila suma sinusoida. Uzrok je bila amplituda $A = 1$, jer visina raste srazmjerno sa $\sqrt{A}$.

Za kalibraciju se koristi činjenica da za slučajnu realizaciju vrijedi $\mathrm{E}|\tilde{h}(\boldsymbol{k}, t)|^2 = 2P_h(\boldsymbol{k})\, w^2(\boldsymbol{k})$, gdje je w težina frekventnog pojasa kaskade ($w = 1$ za cijeli spektar). Varijansa visine u jednoj tački je zato

$$\sigma^2 = 2 \sum_{\boldsymbol{k}} P_h(\boldsymbol{k})\, w^2(\boldsymbol{k}), \tag{5}$$

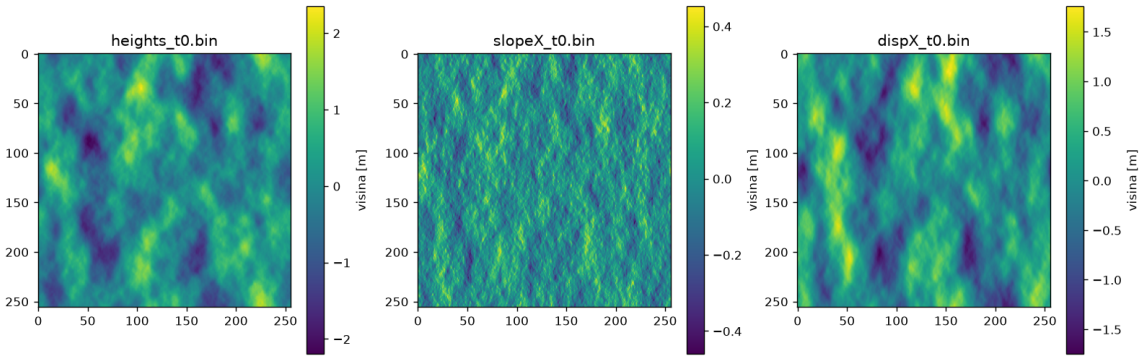
a amplituda koja daje zadanu rms visinu σ_{cilj} je $A = \sigma_{\text{cilj}}^2 / \left(2 \sum P_h|_{A=1}\, w^2\right)$. Ciljna visina se zadaje preko brzine vjetra, uz orijentacionu vezu značajne visine talasa $H_s \approx 0.0214\, V^2$ i $\sigma = H_s/4$. Numerička provjera ($N = 128$, $L = 250\,\text{m}$, $V = 12\,\text{m/s}$, prosjek 40 različitih slučajnih realizacija) dala je teorijski rms $0.73\,\text{m}$ i izmjereni $0.72\,\text{m}$.



Slika 4: Visinsko polje prije kalibracije (amplituda $A = 1$): vrijednosti su u hiljadama metara.

3.1.2 Simultani prikaz polja visine, nagiba i choppy pomaka

Slika 5 prikazuje trostruki pregled izračunatih polja u trenutku $t = 0$ s: vertikalnu visinu h, nagib po x-osi $\partial h/\partial x$, te horizontalni pomak D_x.



Slika 5: Izvezena polja u $t = 0$ s: visina h (lijevo), nagib $\partial h/\partial x$ (sredina) i horizontalni pomak D_x (desno).

- **Visina h (lijevo):** Pokazuje glatku prostornu raspodjelu sa makroskopskim talasnim strukturama.
- **Nagib $\partial h/\partial x$ (sredina):** Sadrži visoke prostorne frekvencije (sitne detalje). Vrijednosti se kreću u opsegu $[-0.4, +0.4]$, što odgovara fizikalno bezdimenzionalnim nagibima okeana.
- **Choppy pomak D_x (desno):** Pokazuje pomake tjemena u rasponu $[-1.5\,\text{m}, +1.5\,\text{m}]$, što omogućava zaoštravanje vrhova bez preklapanja mreže za $\lambda = 1.0$.

3.2 3D Real-Time Renderovanje i Performanse (`viewer.cpp`)

Za 3D prikaz u realnom vremenu koristi se OpenGL 4.3 core profile. Mreža tjemena dimenzija $(N + 1) \times (N + 1)$ ažurira se dinamički u svakom frejmu izračunatim poljima na CPU-u, a zatim šalje u GPU Vertex Buffer Object (VBO) putem `glBufferSubData`.

4 Upravljanje Aplikacijom, Interaktivne Komande i Učinak Modula

Interaktivni vizualizator `viewer_2.cpp` omogućava dinamičku kontrolu fizikalnih i grafičkih parametara u realnom vremenu tokom izvršavanja.



Slika 6: Prikaz izgleda nove verzije za N=256.

4.1 Vodič za Pokretanje i Interaktivne Komande

Aplikacija se pokreće pokretanjem izvršne datoteke `ocean_viewer.exe`. Tokom rada interaktivnog 3D prikaza dostupne su sljedeće kontrole:

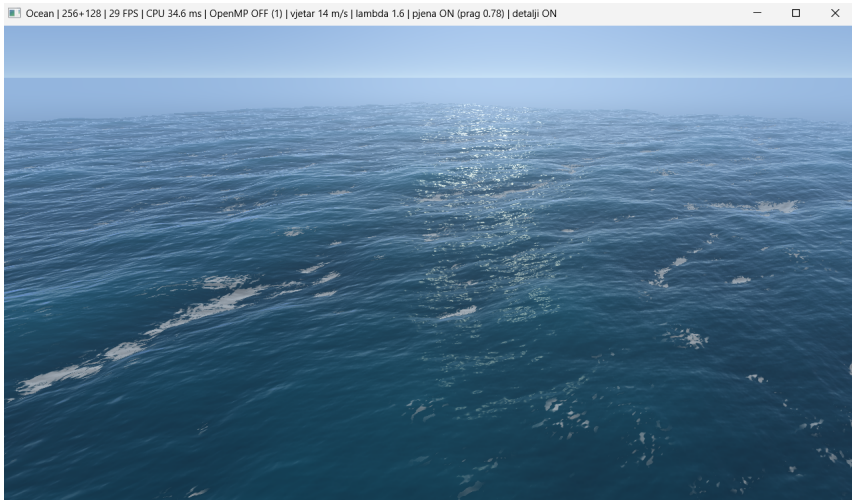
Komanda / Taster	Funkcija i Učinak u Simulaciji
Lijevi Klik + Povlačenje	Rotacija kamere oko centra okeana (promjena yaw i pitch uglova).
Kotačić Miša (Scroll)	Kontrola zumiranja kamere (promjena udaljenosti $gDist$).
Strelica DESNO / LIJEVO	Povećanje / smanjenje brzine vjetra V za ± 1 m/s (automatska re-kalibracija amplitude A).
Strelica GORE / DOLJE	Povećanje / smanjenje choppiness faktora λ za ± 0.1 (zaoštravanje vrhova talasa).
[/] (Uglaste zagrade)	Podešavanje praga nastanka pjene (*foam bias*) u opsegu $[0.3, 1.0]$.
O	Uključivanje / isključivanje OpenMP višenitne paralelizacije u realnom vremenu.
G	Uključivanje / isključivanje simulacije pjene (*Jacobian foam*).
D	Uključivanje / isključivanje visoke kaskade detalja (Kaskada 1 tekstura nagiba).
F	Uključivanje / isključivanje žičanog prikaza (*wireframe mode*).
P	Pauziranje / nastavak proteka vremena simulacije.
ESC	Zatvaranje aplikacije i oslobađanje resursa.

Tabela 1: Pregled interaktivnih komandi u aplikaciji `viewer.cpp`.

4.2 Uticaj ključnih prekidača: OpenMP, pjena i detalji

4.2.1 OpenMP paralelizacija (taster `O`)

- **OpenMP ON (podrazumijevano):** spektar, priprema polja, 2D IFFT, pjena i priprema tjemena izvršavaju se paralelno na 8 niti. Na $N = 256$ računanje frejma pada sa 34.4 ms na 11.5 ms (benchmark), a viewer prikazuje oko 67 FPS (13.8 ms po frejmu uključujući pripremu tjemena).
- **OpenMP OFF:** sve se izvršava na jednom jezgru. U viewer-u je izmjereno oko 35 FPS (30.2 ms po frejmu).



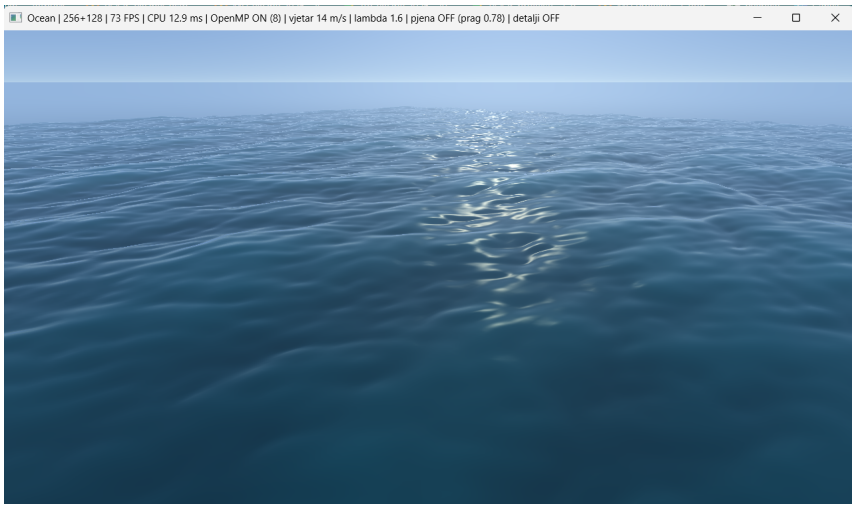
Slika 7: Prikaz izgleda nove verzije za OpenMD OFF

4.2.2 Pjena (taster `G`)

- **Pjena ON:** gdje je Jacobijan J ispod praga (talasi se skupljaju i lome), akumulira se pjena koja postepeno nestaje. U shaderu se prikazuje svijetli sloj pjene čiji rub razbijaju sitni detalji nagiba.
- **Pjena OFF:** isključuje se samo *prikaz* pjene u shaderu. Proračun Jacobijana se i dalje izvodi u svakom frejmu (oko 1.1 ms sekvencijalno za $N = 256$), pa se vrijeme izvršavanja ne mijenja.

4.2.3 Kaskada detalja (taster `D`)

- **Detalji ON:** druga kaskada ($N_1 = 128$, $L_1 = 40$ m) računa sitne nagibe koji se kao `RG32F` tekstura sa mipmapama šalju na GPU. Površina dobija sitne talasiće, a najfiniji razmak tačaka je 0.31 m.
- **Detalji OFF:** ostaje samo geometrijska kaskada ($N_0 = 256$, $L_0 = 400$ m), pa izbliza površina djeluje glađe. Štedi se oko 2.6 ms po frejmu ($N = 256$, sekvencijalno), jer se preskače računanje kaskade 1.



Slika 8: Prikaz izgleda nove verzije kada su pjena i detalji OFF

5 Paralelizacija na CPU (OpenMP)

5.1 Pristup i analiza zavisnosti

Algoritam se paralelizuje podjelom podataka (*data parallelism*): većina koraka izvodi istu operaciju nad N^2 međusobno nezavisnih elemenata mreže. Koristi se OpenMP u verziji koju podržava MSVC (OpenMP 2.0), što znači da brojači paralelnih petlji moraju biti označeni cijeli brojevi (`int`). Globalni prekidač `gUseOmp` u klauzuli `if(gUseOmp)` omogućava da se sekvencijalna i paralelna verzija uporede u istom programu bez dupliranja koda.

Korak	**Vrsta paralelizma**	**Realizacija**
Generisanje $\tilde{h}_0$ (`generateH0`)	sekvencijalno (izvodi se jednom)	slučajni brojevi se generišu redom radi ponovljivosti
Spektar $\tilde{h}(\boldsymbol{k}, t)$ (`spectrumAtTime`)	svaka tačka $\boldsymbol{k}$ je nezavisna	`omp parallel for` po redovima; svaka iteracija piše samo u svoj element
Priprema spektara polja (nagibi, pomaci)	svaka tačka je nezavisna	`omp parallel for`
2D IFFT, prolaz po redovima	svaki red je nezavisan	`omp for`, privatni niz `line` po niti
2D IFFT, prolaz po kolonama	svaka kolona je nezavisna, ali zavisi od cijelog prethodnog prolaza	`omp for` nakon implicitne barijere
Tri IFFT-a (visina, nagibi, pomaci)	međusobno nezavisni (paralelizam zadataka)	izvode se jedan za drugim; paralelizam je unutar svakog IFFT-a
Realni dio i skaliranje	svaki element je nezavisan	`omp parallel for` (u `computeSlopesRG` završna petlja je sekvencijalna)
Pjena (`updateFoam`)	svaka tačka je nezavisna (čitaju se polja pomaka, piše se samo u svoj element)	`omp parallel for`
Priprema tjemena (viewer)	svako tjeme je nezavisno	`omp parallel for`
Alokacija pomoćnih nizova	sekvencijalno	konstruktori `std::vector`

Tabela 2: Gdje se algoritam može paralelizovati i kako je to realizovano.

5.2 Realizacija 2D IFFT-a

Dva prolaza (redovi, pa kolone) smještena su u jedan paralelni region, tako da se tim niti pokreće samo jednom:

```
void fft2d(std::vector<cd>& a, size_t nn, bool inverse) {
    const int n = int(nn);
    #pragma omp parallel if(gUseOmp)
    {
        std::vector<cd> line(nn);      // privatni niz svake niti
        #pragma omp for
        for (int r = 0; r < n; ++r) { /* red r: kopiraj, fft, vrati */ }
        // implicitna barijera: kolone smiju poceti tek kad su svi redovi gotovi
        #pragma omp for
        for (int c = 0; c < n; ++c) { /* kolona c: kopiraj, fft, vrati */ }
    }
}
```

Dvije stvari su ključne za ispravnost. Prvo, svaka nit mora imati **vlastiti** niz `line`; kad bi on bio zajednički, niti bi međusobno prepisivale podatke (*race condition*). Drugo, između prolaza po redovima

i po kolonama postoji **barijera**, jer kolonski prolaz čita rezultate svih redova. Kad je `gUseOmp` netačan, region se izvršava sa jednom niti, pa se dobija sekvencijalna verzija iste logike.

5.3 Verifikacija ispravnosti

Rezultati paralelne verzije su upoređeni sa sekvencijalnom ($N = 256$, 8 niti). Najveća relativna razlika je 0 za osnovnu varijantu (5 IFFT), za pakovanu varijantu (3 IFFT), za pjenu i za kaskadu detalja. Pakovana varijanta se od osnovne razlikuje najviše za $8.1 \cdot 10^{-7}$ (relativno), a visina je identična.

5.4 Metodologija mjerenja

Mjerenja su izvedena programom `bench.cpp` na računaru sa procesorom [model procesora] (4 fizička jezgra, 8 logičkih procesora), operativnim sistemom Windows 11 Pro i kompajlerom MSVC iz Visual Studija 2026, u *Release* konfiguraciji, bez debuggera, bez drugih pokrenutih programa i bez štednje energije. Za svaku kombinaciju veličine i načina izvršavanja izvodi se 20 zagrijavanja i 30 mjerenja, a prijavljuje se medijan. Prvi prolaz kroz sve konfiguracije služi samo za zagrijavanje procesora i ne prijavljuje se. Mjerenje je ponovljeno u tri serije.

5.5 Rezultati i analiza

Broj niti	Ubrzanje (medijan 3 serije)	Raspon	Efikasnost
2	$1.74\times$	1.70–$1.88\times$	87%
4	$2.50\times$	2.47–$2.92\times$	62%
8	$4.25\times$	4.17–$4.33\times$	53%

Tabela 3: Ubrzanje osnovne varijante (5 IFFT) za $N = 256$ naspram sekvencijalnog izvršavanja.

- **IFFT dominira.** Za $N = 256$ čini oko 93% sekvencijalnog vremena, a na 8 niti još oko 87%, pa je on glavna meta paralelizacije.
- **Ubrzanje na 8 niti je** 4.1–$4.6\times$ za sve tri veličine ($N = 128$: 4.1–$4.3\times$, $N = 256$: 4.2–$4.3\times$, $N = 512$: 4.2–$4.6\times$), uz efikasnost od oko 50–57% po niti.
- **Odnos prema fizičkim jezgrima.** Procesor ima 4 jezgra i 8 logičkih procesora. Ubrzanje od oko $4.2\times$ na 8 niti veće je od broja jezgara, što znači da simultano višenitno izvršavanje (SMT) donosi dodatnu dobit. Naspram 4 fizička jezgra efikasnost je oko 105%, pa vrijednost od 53% po logičkoj niti ne znači slabu paralelizaciju.
- **Prelaz sa 2 na 4 niti** daje samo oko $1.4\times$ dodatnog ubrzanja (idealno $2\times$). Mogući uzroci su raspored niti na logičke procesore istog jezgra, pristup memoriji u prolazu po kolonama (veliki korak, dijeljenje keš linija među nitima) i pad frekvencije pri većem opterećenju. Ovo su hipoteze koje nisu potvrđene; planirana je provjera mjerenjem prolaza po redovima i po kolonama te vezivanjem niti za logičke procesore.
- **Priprema i izvlačenje polja se slabo ubrzavaju.** Vjerovatno zato što troše vrijeme na alokaciju i kopiranje velikih nizova (a ne na računanje), što se izvodi sekvencijalno. Njihov udio je mali, pa ne ograničavaju ukupno ubrzanje, ali ga ne poboljšavaju.
- **Za malo N ubrzanje je slabije.** Za $N = 128$ ukupni frame (sa pjenom i kaskadom detalja) ubrzava se samo $3.9\times$, jer je posao mali, a kaskada detalja je fiksne veličine (128×128).
- **Varijabilnost mjerenja.** Među serijama se rezultati razlikuju; npr. za pakovanu varijantu na 8 niti ($N = 256$) ukupno vrijeme je bilo između 6.7 ms i 10.0 ms. Zato se prijavljuju medijan i raspon više serija.

5.6 Ograničenja i moguća poboljšanja

- pomoćni nizovi se alociraju u svakom frejmu; ponovna upotreba bafera bi smanjila sekvencijalni dio,
- prolaz po kolonama ima loš obrazac pristupa memoriji; obrada blokova kolona ili transponovanje matrice bi poboljšali keš lokalnost,
- twiddle faktori se računaju funkcijom `std::polar` u svakom leptiru; unaprijed izračunata tablica bi smanjila trošak,
- tri IFFT-a se izvode jedan za drugim, iako su međusobno nezavisni,
- završna petlja u `computeSlopesRG` je sekvencijalna.

6 Eksperimentalni rezultati i profilisanje

Rezultati poređenja osnovne (5 IFFT), pakovane (3 IFFT) i kompletne (`frame`) varijante prikazani su u tabeli 4. Sva vremena su u milisekundama (medijan 30 mjerenja jedne serije); *Ubrz.* je ubrzanje naspram iste varijante sekvencijalno, a *vs osn.* naspram osnovne sekvencijalne varijante.

N	Varijanta	Niti	Spektar	Prip.	IFFT	Izvl.	Pjena	Detalji	Ukupno	Ubrz.	vs osn.
128	5 IFFT	1 (seq)	0.60	0.38	9.67	0.17	-	-	10.86	1.00×	1.00×
128	5 IFFT	8	0.15	0.30	1.98	0.14	-	-	2.64	4.12×	4.12×
128	3 IFFT	1 (seq)	0.57	0.19	5.78	0.06	-	-	6.67	1.00×	1.63×
128	3 IFFT	8	0.13	0.09	1.21	0.03	-	-	1.58	4.23×	6.88×
128	frame	1 (seq)	0.56	0.19	5.80	0.06	0.27	2.54	9.70	1.00×	1.12×
128	frame	8	0.12	0.07	1.19	0.05	0.07	0.59	2.50	3.88×	4.35×
256	5 IFFT	1 (seq)	2.24	0.98	45.74	0.34	-	-	49.30	1.00×	1.00×
256	5 IFFT	8	0.51	0.61	10.29	0.43	-	-	11.82	4.17×	4.17×
256	3 IFFT	1 (seq)	2.28	0.84	27.45	0.26	-	-	30.87	1.00×	1.60×
256	3 IFFT	8	0.81	0.45	8.40	0.33	-	-	10.01	3.08×	4.92×
256	frame	1 (seq)	2.24	0.80	27.37	0.28	1.10	2.57	34.43	1.00×	1.43×
256	frame	8	0.80	0.42	8.53	0.28	0.34	1.06	11.46	3.00×	4.30×
512	5 IFFT	1 (seq)	9.44	5.53	217.42	2.26	-	-	235.08	1.00×	1.00×
512	5 IFFT	8	2.31	4.02	45.79	3.44	-	-	55.60	4.23×	4.23×
512	3 IFFT	1 (seq)	9.51	4.04	127.26	2.03	-	-	143.08	1.00×	1.64×
512	3 IFFT	8	2.31	2.42	27.37	3.38	-	-	35.64	4.01×	6.60×
512	frame	1 (seq)	9.44	4.01	127.66	1.99	4.40	2.61	150.41	1.00×	1.56×
512	frame	8	2.21	2.51	27.20	3.46	1.14	0.61	37.72	3.99×	6.23×

Tabela 4: Poređenje varijanti po koracima i ostvarena ubrzanja (jedna serija mjerenja). Kolona *Ukupno* je medijan ukupnog vremena po ponavljanju, pa se može neznatno razlikovati od zbira medijana pojedinih kolona.

Pakovanje smanjuje vrijeme IFFT-a za oko 40% (5 transformacija zamijenjeno sa 3). Pjena i kaskada detalja zajedno čine oko 29% sekvencijalnog vremena frejma za $N = 128$, oko 11% za $N = 256$ i oko 5% za $N = 512$, jer je kaskada detalja fiksne veličine (128×128). Za $N = 256$ novi frame sa svim efektima traje $34.4\,\text{ms}$ sekvencijalno i $11.5\,\text{ms}$ na 8 niti, dok je prva verzija trajala $49.3\,\text{ms}$ i $11.8\,\text{ms}$, dakle uz više posla nije sporija.

7 Ograničenja trenutne implementacije

- **Efektivna rezolucija.** Razmak tačaka od $0.31\,\text{m}$ (ekvivalent mreže od oko 1280×1280) vrijedi samo za *nagibe*, odnosno osvjetljenje. Geometrija površine ostaje na razmaku od $1.56\,\text{m}$, a kaskada detalja se ponavlja svakih $40\,\text{m}$.

- **Pjena.** Pravi uvjet loma talasa ($J < 0$) pri umjerenom λ gotovo se nikad ne ispuni, pa se prag pjene (ovdje 0.78) bira empirijski. Pjena se čuva po tački mreže i kreće se zajedno s tjemenom. Fizički lom i prskanje nisu modelovani.
- **Vizuelni model.** Nema refrakcije ni providnosti vode.
- **Hardver.** Simulacija se u cijelosti računa na CPU-u; OpenGL se koristi samo za crtanje.
- **Alokacije.** Pomoćni nizovi se alociraju u svakom frejmu (vidi odjeljak 5.6).

8 Zaključak

Poboljšana verzija koristi pakovani IFFT (3 umjesto 5 transformacija po frejmu), pjenu izračunatu preko Jacobijana i dodatnu kaskadu sitnih detalja (128^2, patch 40 m), čime se najfiniji razmak tačaka za osvjetljenje smanjuje na 0.31 m. Za $N = 256$ sekvencijalno vrijeme frejma smanjeno je sa 49.3 ms na 34.4 ms ($1.43\times$), a na 8 niti ostalo je približno isto (11.8 ms naspram 11.5 ms), iako nova verzija računa znatno više.

OpenMP paralelizacija daje ubrzanje od oko 4.1–$4.6\times$ na 8 niti (4 fizička jezgra) i rezultate identične sekvencijalnoj verziji. IFFT čini oko 93% sekvencijalnog vremena, pa je on glavna meta daljeg ubrzanja. Ograničenja su navedena u prethodnom odjeljku. Sljedeći korak je prebacivanje računanja na grafičku karticu pomoću compute shadera.